Containerization Fundamental

- Amjad Hossain
12th April, 2026

Docker Desktop (Windows & macOS)

Official Download Page

- https://www.docker.com/products/docker-desktop/

Official Docs

- https://docs.docker.com/desktop/

Covers:

- Docker Engine
- Docker CLI
- Docker Compose (v2)
- GUI Dashboard
- Kubernetes (optional)

Basic Dockerfile Example

```
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app

COPY target/app.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

FROM

- base image (OS + runtime)

WORKDIR

- working directory inside container

COPY

- copies files into image

EXPOSE

- documents container port

ENTRYPOINT

- defines startup command

Basic Docker Commands

Build Command

```
docker build -t my-app:1.0 .
```

- `-t` → tag
- `.` → current directory (build context)

Run Command

```
docker run -p 8080:8080 my-app:1.0
```

- maps port
- starts container

Don’t forget

.dockerignore

Java Spring Boot Example

```
target/
*.log
*.tmp
.git
.gitignore
Dockerfile
docker-compose.yml
.env
```

Docker Linting

1. https://hadolint.github.io/hadolint/
2. https://www.fromlatest.io/#/

Dockerfile

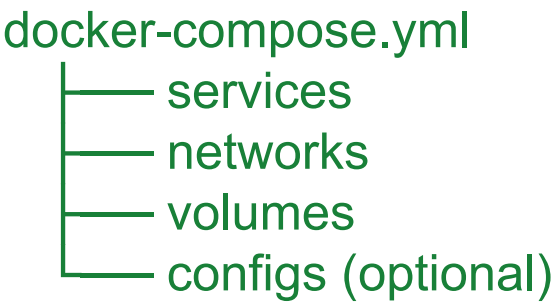
```
# Build Stage
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /build
COPY ..
RUN mvn clean package -DskipTests
```


```
# Runtime Stage
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /build/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java",`jar app.jar"]
```

Build & Run Spring Boot App Using Dockerfile

Docker Compose

High-Level View



Each part has a clear responsibility

Docker Compose: **Service**

Defines **containers**

Example:

services:
  user-service:
  db:
  kafka:

Responsibility:

- what to run
- how to run
- configuration of container

Each service = one running container

Docker Compose: **Network**

Controls **communication between services**

```
networks:
 app-network
```

Responsibility:

- connect containers
- enable service-to-service communication

Docker Compose: **Volumes**

Handles **data persistence**

```
volumes:
 db-data:
```

Used in service:

```
db:
 volumes:
   - db-data:/var/lib/postgresql/data
```

Responsibility:

- store data outside container lifecycle

Docker Compose: **Ports**

```
ports:
 - "8080:8080"
```

Responsibility:

- expose container to host

Format:

host_port : container_port

Docker Compose: **Environment**

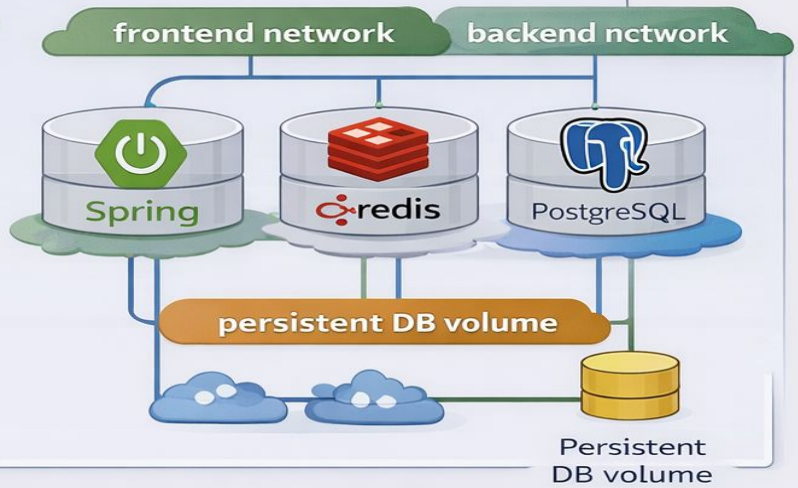
```
environment:
    DB_HOST: db
    DB_PORT: 5432
```

Responsibility:

- inject runtime configuration

docker-compose.yml

```
version: "3.9"
services:
  app: my-spring-app
  - networks:
      - backend
      - frontend
 - redis:
  - image: redis:7
  -  networks:
    -image: apache/kafka:3.5
    networks:
  db:
  - image: postgres:15
  ---volumes:
      - db-data:/var/lib
        postgresql/data
  - networks:
volumes:
db-data:
networks:
  frontend:
  backend
```



Volume and Network Configuration

Kuberenetes lab preparation

For a basic EKS lab, these are the main AWS services we need:

- **AWS account** and billing enabled. AWS Free Tier currently offers new customers credits and a limited free plan window, but EKS itself is not generally “free” in the same sense as simple free-tier services. Ref link
- **Amazon EKS** for the Kubernetes control plane. AWS charges **$0.10 per cluster hour** while the Kubernetes version is in standard support, and a higher rate if you let it drift into extended support. Ref link
- **Compute for worker nodes**, typically **EC2 On-Demand instances** for a lab. EC2 On-Demand is pay-as-you-go with no long-term commitment. Ref link
- **Amazon ECR** to store your Docker images. ECR charges for storage and certain data transfer/image operations. Ref link
- **VPC networking**. In practice EKS needs VPC/subnets, and if you use private subnets with outbound internet access, **NAT Gateway** can become a noticeable cost because AWS charges both hourly and per-GB processing. Ref link
- **Load balancing / ingress**, usually via the **AWS Load Balancer Controller**, which AWS recommends installing with Helm if you are new to EKS. Ref link
- Managed database: https://aws.amazon.com/rds/pricing/

Thank you